```
        if (++fade > 255) {
            fade = 255;
            going_up = false;
        }
    } else {
        if (--fade < 0) {
            fade = 0;
            going_up = true;
 }
    }
    pwm_set_gpio_level(PICO_DEFAULT_LED_PIN, fade * fade);
}

int main() {
    fade = 0;
    going_up = 0;
    gpio_set_function(PICO_DEFAULT_LED_PIN, GPIO_FUNC_PWM);
    slice = pwm_gpio_to_slice_num(PICO_DEFAULT_LED_PIN);
    pwm_clear_irq(slice);
    pwm_set_irq_enabled(slice, true);
    irq_set_exclusive_handler(PWM_IRQ_WRAP, on_pwm_wrap);
    irq_set_enabled(PWM_IRQ_WRAP, true);
    struct {
        int csr, div, top;
    } config;
    memcpy((int)&config, (int)pwm_get_default_config(),
sizeof(config));
    pwm_config_set_clkdiv((int)&config, 4.0);
    pwm_init(slice, (int)&config, true);
    while (1) {
        wfi();
        if (getchar_timeout_us(500000) == 3)
            break;
    }
    irq_set_enabled(PWM_IRQ_WRAP, false);
    pwm_set_irq_enabled(slice, false);
    return 0;
}
```

Lightweight UNIX-style shell

Our next tool is a lightweight, real-time command-line interface (CLI) for the Raspberry Pi Pico W 2 (RP2350 dual-core), built entirely in MicroPython. This shell provides useful commands for Wi-Fi networking with persistent config, telnet daemon, file system create/read/delete/navigate, script runner and file downloader, memory/clock speed/device information, and overclocking tools.

First, flash your Pico W board with the appropriate MicroPython firmware. Now create the *Dockerfile_PicoShell* shown below and build a container image that has all the artifacts required by using the *docker build -f Dockerfile_PicoShell . -t picoshell* command. Next, execute the commands sequence given below on your machine to transfer the necessary files to your Pico board (adjust for the correct port on which your Pico board connected to your machine).

```
FROM mpremote
SHELL ["/bin/ash", "-o", "pipefail", "-c"]
RUN wget https://github.com/patrickp02/PicoShell/archive/
refs/heads/main.zip \
  && unzip main.zip \
  && rm main.zip

WORKDIR /etc/rpipico/PicoShell-main

for f in {boot,main,utils}.py telnet docs config.txt
do
  docker run --rm --device=/dev/ttyACM1:/dev/ttyACM1 \
    picoshell connect auto fs cp -r "${f}" :
done
```

Now restart and connect to your Pico W board over a serial link, as we did earlier. You should be presented with a colourful shell prompt. Executing the *help* command on the shell should dump a help screen as shown in Figure 2.

The *blink* command flashes your Pico W onboard LED a few times. You can dump board level information through the *ram*, *dspace* and *sysinfo* commands. Executing the *scan* command will show up the 2.4GHz Wi-Fi networks available around your board. To connect with a wireless network, enter the *wifi* command and give the network ID and password. You could dump the internal onchip temperature reading using the *temp* command. The *ls* command dumps the listing of files and directories on your Pico W board root. The PicoShell also provides file system commands to create/delete directory, remove file and read file. You can find the current clock speed using the *clock* command and set it with the command *setclock*. You can download a Python or text file with the *clone* command and run Python scripts from storage using the *run* command. Popular *curl*, *ping* and *pmap* commands are also provided by PicoShell.

Issuing the *exit* command drops you to the MicroPython REPL prompt. Running the code chunk *from machine import soft_reset;soft_reset()* on the MicroPython prompt relaunches the PicoShell. Adding more commands in the PicoShell is easy. Just add your Python routines in *utils.py*, update *main.py* for linking the commands with routines, copy the updated *utils.py, main.py* to the board and restart your board. PicoShell is definitely a handy tool in your Pi Pico journey.

Getting retro with a DOS-like shell

Are you ready to experience the good old classic computing days? PyDOS-shell is a Python layer created over Circuit/MicroPython covering multiple microcontroller boards. I used